Hotel Du Soleil — Documentation

This folder contains the full technical documentation for the **Hotel Du Soleil** website (`www.hotel-du-soleil.it`), a multilingual marketing site for an alpine hotel in Torgnon, Valle d'Aosta, built with **SvelteKit 2 + Svelte 5 + Tailwind CSS 4**.

Contents

Quick links

- **Production site:** https://www.hotel-du-soleil.it
- **Booking engine:** https://booking.slope.it
- **Framework docs:** SvelteKit, Svelte 5 runes, Tailwind CSS 4

> New to the codebase? Start with Architecture, then skim
> Project structure and the Development guide.

analytics-and-consent

Analytics & cookie consent

The site uses **Google Analytics 4** (measurement ID `G-GGKFG688CK`) with **Google Consent Mode v2**, gated behind an explicit cookie banner. Analytics events only fire
after the visitor accepts cookies.

Pieces

File	Role
`src/app.html`	Loads `gtag.js`, sets Consent Mode **defaults to denied**, and configures GA with `send_page_view: false` + `anonymize_ip: true`.
`src/lib/consent.ts`	Consent store + `localStorage` persistence + Google consent updates.
`src/lib/components/CookieBanner.svelte`	The accept/reject UI.
`src/lib/analytics.ts`	Consent-aware event helpers + URL classifiers.
`src/routes/+layout.svelte`	Initializes consent and wires up page-view, scroll-depth, and click tracking.

Consent flow

1. **Default denied.** `app.html` calls `gtag('consent', 'default', { analytics_storage: 'denied', ad_*: 'denied', ... })` before anything

else, and configures GA with `send_page_view: false` so nothing is sent automatically.

2. **Initialize on load.** The layout calls `initializeCookieConsent()` (from `consent.ts`) on mount, which reads `localStorage['hds_cookie_consent']`:
   - `'accepted'` → store set to accepted, Google consent updated to `analytics_storage: 'granted'`.
   - `'rejected'` → store rejected, consent stays denied.
   - unset → the `CookieBanner` is shown (`$cookieConsent === null`).
3. **User choice.** `setCookieConsent('accepted' | 'rejected')` persists the choice and calls `updateGoogleConsent(...)`. Ads-related signals are always denied; only `analytics_storage` toggles.
4. **Reset.** `resetCookieConsent()` clears the stored choice (re-shows the banner) — useful from a cookie-policy "manage preferences" control.

```
// consent.ts (storage key + states)
const STORAGE_KEY = 'hds_cookie_consent';
export type CookieConsent = 'accepted' | 'rejected' | null;
export function hasAnalyticsConsent(): boolean {
	return isBrowser() && localStorage.getItem(STORAGE_KEY) ===
'accepted';
}
```

Event tracking

`src/lib/analytics.ts` exposes helpers that **no-op unless consent is granted** and `window.gtag` exists:

Helper	Emits	When
`trackPageView(path, title)`	`page_view`	On every client-side navigation (from the layout's `$effect`).
`trackEvent(name, params)`	custom event	Used by the tracking listeners below.
`isBookingUrl(url)`	—	True for `booking.slope.it` links.

Helper	Emits	When
`classifyContactHref(href)`	`'phone' | 'email' | 'whatsapp' | 'other' | null`	Classifies `tel:` / `mailto:` /WhatsApp links.

The root layout installs a capturing document click listener and a scroll listener that emit:

Event	Trigger	Key params
`page_view`	route change	`page_path`, `page_title`, `page_location`
`scroll_depth`	reaching 25/50/75/90% of the page	`percent_scrolled`, `page_path`
`contact_click`	clicking a `tel:` / `mailto:` /WhatsApp link	`contact_type`, `link_url`, `page_path`
`begin_checkout`	clicking a link to the booking engine	`currency: 'EUR'`, `link_url`, `page_path`
`select_promotion`	clicking a link to `/offerte/<slug>`	`promotion_id`, `page_path`
`outbound_click`	clicking any cross-origin link	`link_url`, `link_domain`, `page_path`

Scroll milestones are tracked per page (the set is cleared on navigation).

Third-party widget

The layout also injects the **Canary** chat widget (`hotel-du-soleil-torgnon83`) on mount. It is loaded from Canary's CDN and themed to the site's gold/charcoal palette.

Adding a new tracked event

1. Add a `trackEvent('your_event', { ... })` call where the interaction happens (or

extend the layout's central click listener for global behaviors).
2. Remember it will only fire when the user has accepted cookies — test by accepting
in the banner first.

architecture

Architecture

Overview

Hotel Du Soleil is a **content/marketing website** for an alpine hotel. It is a SvelteKit application that is mostly static (pages are pre-rendered/SSR'd marketing
content) with a few dynamic touches: a server-rendered weather page, client-side internationalization, analytics, and deep-links into an external booking engine.

There is **no application database and no custom backend API**. All "data" is either:

- hard-coded in TypeScript modules (e.g. room definitions in `src/lib/rooms.ts`),
- stored as translation JSON under `src/lib/i18n/locales`, or
- fetched from third-party services at request/render time (Open-Meteo, Cloudinary, the Slope booking engine, Canary chat, Google Analytics).

Tech stack

Layer	Technology
Framework	SvelteKit `^2.50`
UI library	Svelte 5 `^5.51` (uses **runes**: `$state`, `$props`, `$derived`, `$effect`, `$bindable`)
Styling	Tailwind CSS 4 via `@tailwindcss/vite`, with a custom theme in `src/routes/layout.css`
Build tool	Vite 7
Language	TypeScript 5 (strict)

Layer	Technology
Deploy adapter	`@sveltejs/adapter-auto` (Netlify in production)
Icons	`lucide-svelte`
Animation	GSAP, `svelte-motion`, native Svelte transitions, CSS + `IntersectionObserver`
Carousels	Swiper and custom carousel components
Media CDN	Cloudinary
Utilities	`clsx` + `tailwind-merge` (the `cn()` helper found in several components)

Rendering model

- **File-based routing.** Everything under `src/routes` maps to a URL. `+page.svelte` files are pages, `+layout.svelte` wraps all pages, `+page.server.ts` runs server-side loaders, and `+server.ts` files are endpoints. See Routes & pages.
- **Adapter-auto.** The build target is detected automatically; in production it builds for **Netlify** (see `netlify.toml` and Deployment).
- **Runes mode** is enabled for project code (not `node_modules`) in `svelte.config.js`.

Request / render lifecycle

1. **Server hook** — `src/hooks.server.ts` runs on every request. It redirects "hidden" sections (`/posizione`, `/sport`, `/sport/*`) to `/` with a `307`, so those pages exist in the repo but are not publicly reachable.
2. **HTML shell** — `src/app.html` is the document template. It sets up async Google Fonts (Cormorant Garamond + Inter), bootstraps **Google Analytics with Consent Mode defaulting to denied**, and injects the SvelteKit head/body.
3. **Root layout** — `src/routes/+layout.svelte` wraps every page with the `Navbar`, `Footer`, `CookieBanner`, `PromoCarousel`, `CloudinaryRuntime`, and `SecurityGuard`. It also:

- initializes cookie consent,
- injects the **Canary** chat widget,
- wires up analytics (page views, scroll-depth, contact/outbound/booking click tracking),
- sets `<html lang>` / `dir` from the active locale,
- runs an `IntersectionObserver` to add `is-visible` for scroll-reveal animations.

4. **Page** — the matched `+page.svelte` renders its content, pulling copy from the
   i18n store and assets from `/imgs/...` (optionally rewritten to Cloudinary).

Cross-cutting concerns

Concern	Where it lives	Doc
Translations	`src/lib/i18n/*`	i18n
Booking links	`src/lib/config/booking.ts`	Booking
Media delivery	`src/lib/cloudinary.ts`, `src/lib/server/cloudinary.ts`, `CloudinaryRuntime.svelte`	Cloudinary
Analytics	`src/lib/analytics.ts`, `+layout.svelte`, `app.html`	Analytics & consent
Consent	`src/lib/consent.ts`, `CookieBanner.svelte`	Analytics & consent
Weather	`src/routes/meteo/*`, `*WeatherWidget.svelte`	Weather
SEO	`src/routes/sitemap.xml/+server.ts`, per-page `<svelte:head>`	Routes

Data-flow diagram (text)

```
Browser request
   │
   ▼
hooks.server.ts ──(hidden path?)──▶ 307 redirect to /
   │ no
```

```
   ▼
app.html (fonts, GA consent=denied default)
   │
   ▼
+layout.svelte ──► Navbar / Footer / CookieBanner / PromoCarousel
   │                 + analytics listeners + Canary chat +
Cloudinary rewrite
   ▼
+page.svelte ──► i18n store ($t)  ──► locale JSON
             ──► /imgs/... assets ──► (optional) Cloudinary CDN
             ──► booking buttons   ──► getBookingEngineUrl() ──►
Slope.it
             ──► weather widgets   ──► Open-Meteo / wttr.in
```

booking

Booking integration

The site does not process bookings itself. All "Book now" actions deep-link into an external booking engine hosted by **Slope.it**. The URLs and promotion IDs are centralized in `src/lib/config/booking.ts`.

Booking engine URL

```
export const BOOKING_ENGINE_URL =
'https://booking.slope.it/140f49cb-e4f4-40e9-b494-25cfa4618e56';

export function getBookingEngineUrl(source?: string): string {
	if (!source) return BOOKING_ENGINE_URL;
	const url = new URL(BOOKING_ENGINE_URL);
	url.searchParams.set('utm_source',
'hotel_du_soleil_website');
	url.searchParams.set('utm_medium', 'referral');
	url.searchParams.set('utm_campaign', source);
	return url.toString();
}
```

- Call `getBookingEngineUrl()` with **no argument** for a bare link.
- Pass a `source` string to attach **UTM parameters** for campaign attribution. The `source` becomes `utm_campaign`. Existing call sites use sources like `booking_bar`, `room_booking_widget`, and `promo_marquee_restart`.

Promotion deep-links

Promotions point at a `/promotions/<id>` path on the booking engine. `getPromotionUrl` composes that path while preserving any UTM params:

```
export function getPromotionUrl(promotionId: string, source?:
string): string {
	const promotionPath = `/promotions/${promotionId}`;
	const base = source ? getBookingEngineUrl(source) :
BOOKING_ENGINE_URL;
	const url = new URL(base);
	url.pathname = `${url.pathname.replace(/\/$/,
'')}${promotionPath}`;
	return url.toString();
}
```

Defined promotions

Helper	Promotion ID constant
`getRestartPromotionUrl(source?)`	`RESTART_PROMOTION_ID`
`getTorgnonHikingAdventurePromotionUrl(source?)`	`TORGNON_HIKING_ADVENTURE_PROM`
`getForteBardGourmetEscapePromotionUrl(source?)`	`FORTE_BARD_GOURMET_ESCAPE_PRO`
`getAostaRomanaPromotionUrl(source?)`	`AOSTA_ROMANA_CASTELLO_DI_FENI`

`getAostaRomanaPromotionUrl` falls back to the bare booking URL if its promotion ID
is empty — a useful pattern when adding a new promotion before its ID is known.

Where booking links are used

Component	Source tag
`BookingBar.svelte`	`booking_bar`
`RoomBookingWidget.svelte`	`room_booking_widget`
`BookingDrawer.svelte`	via `getBookingEngineUrl`
`PromoCarousel.svelte`	`promo_marquee_*` (per promotion)

The booking widgets (`BookingBar`, `RoomBookingWidget`, `BookingDrawer`) collect
arrival/departure dates and guest counts locally via the shared `Calendar` component,
then hand off to the engine URL. Direct contact options (phone/WhatsApp/email) are
also offered in `BookingDrawer` and `OfferLeadForm`.

Analytics tie-in

Clicks that navigate to `booking.slope.it` are detected by `isBookingUrl()` in `src/lib/analytics.ts` and tracked as a GA `begin_checkout`
event; links to `/offerte/<slug>` fire `select_promotion`. See
Analytics & consent.

Adding a new promotion

1. Add the promotion ID constant and a `getXyzPromotionUrl()` helper to `src/lib/config/booking.ts`.
2. Create the offer page under `src/routes/offerte/<slug>/+page.svelte`.
3. Link the page/CTA to the new helper, passing a descriptive `source`.
4. Add the offer to `OffersCarousel` / `PromoCarousel` if it should be featured.

5. Add the route to the sitemap (`src/routes/sitemap.xml/+server.ts`).

cloudinary

Cloudinary media pipeline

Images and videos can be served either from the local `static/imgs/` folder or from
Cloudinary (a media CDN with on-the-fly transformations). The integration is designed to be **opt-in and non-breaking**: with no configuration the site keeps using
local assets, and any unmapped asset falls back to its local file.

Pieces

File	Role
`src/lib/cloudinary.ts`	Browser-safe URL building, config flags, manifest lookup, unsigned-upload helpers. Re-exported from `$lib`.
`src/lib/server/cloudinary.ts`	Server-only signed operations (uses the API secret).
`src/lib/components/CloudinaryRuntime.svelte`	Runtime DOM hook that rewrites `/imgs/...` URLs to Cloudinary when enabled.
`src/lib/generated/cloudinary-manifest.json`	Generated map of local paths → Cloudinary `{ publicId, resourceType, version, ... }`.
`scripts/sync-cloudinary-manifest.mjs`	Builds the manifest from your Cloudinary account.

Environment variables

Set these in `.env` (template in `.env.example`):

Variable	Scope	Meaning
`PUBLIC_CLOUDINARY_CLOUD_NAME`	public	Required for any delivery URL.
`PUBLIC_CLOUDINARY_API_KEY`	public	Required for unsigned browser uploads.
`PUBLIC_CLOUDINARY_UPLOAD_PRESET`	public	Required for unsigned browser uploads.
`PUBLIC_CLOUDINARY_ENABLE_DELIVERY`	public	`'true'` turns on automatic rewriting of local `/imgs/...` paths. Default `false`.
`PUBLIC_CLOUDINARY_BASE_FOLDER`	public	Folder prefix used when mapping local assets to public IDs (default `hotel-du-soleil`).
`CLOUDINARY_API_SECRET`	**server-only**	Used for signed uploads/admin operations. Never sent to the client.

Derived config flags exposed by `cloudinary.ts`:

- `hasCloudinaryDelivery` — a cloud name is set.
- `isCloudinaryDeliveryEnabled` — cloud name set **and** delivery flag `true`.
- `hasCloudinaryUnsignedUpload` — cloud name + API key + upload preset are set.

Building delivery URLs

```
import { getCloudinaryDeliveryUrl, resolveCloudinaryUrl } from
'$lib';

// Explicit public ID + transformations
const heroUrl = getCloudinaryDeliveryUrl('hotel-du-
soleil/home/hero', 'image', {
	width: 1600,
	height: 900,
	crop: 'fill',
	quality: 'auto',
	format: 'auto'
});
```

```
// Resolve a value that might be a local path, a remote URL, a
public ID, or an object
const src = resolveCloudinaryUrl({
       publicId: 'hotel-du-soleil/rooms/matrimoniale-hero',
       fallbackSrc: '/imgs/Rooms/matrimoniale-superior-hero-
1.webp'
});
```

`getCloudinaryDeliveryUrl(publicId, resourceType, options, config)` assembles `https://res.cloudinary.com/<cloud>/<type>/upload/<transforms>/v<version>/<publicId>`.
The supported transformation options map to Cloudinary's URL params (e.g. `width` → `w_`, `crop` → `c_`, `quality` → `q_`, `gravity` → `g_`, plus `custom: string[]` for raw segments).

`resolveCloudinaryUrl` resolution order

1. A full `http(s)://` string → returned unchanged.
2. A `/...` path → only rewritten if delivery is enabled **and** it's a `/imgs/...` asset present in the manifest; otherwise the local path is returned.
3. A bare string (treated as a public ID) → a delivery URL if a cloud name is set.
4. An object with `publicId` → a delivery URL (or `fallbackSrc` when delivery is off).
5. Otherwise `fallbackSrc`, or throws if neither is available.

Automatic runtime rewriting

`CloudinaryRuntime.svelte` is mounted globally in the root layout. When `PUBLIC_CLOUDINARY_ENABLE_DELIVERY=true`, it walks the DOM and rewrites `img[src]`, `source[src]`, and `poster` attributes through `resolveCloudinaryUrl`, and uses a `MutationObserver` to handle nodes added

later. This means existing pages keep using `/imgs/...` markup unchanged — delivery
is swapped in transparently. Assets not in the manifest fall back to the local file.

Manifest sync workflow

After uploading new files to Cloudinary, regenerate the manifest so the runtime can
map local paths to real public IDs/versions:

```
npm run cloudinary:sync
```

`scripts/sync-cloudinary-manifest.mjs`:

1. Loads Cloudinary credentials from `.env` (throws if missing).
2. Fetches **all** image and video resources from your Cloudinary account.
3. Walks `static/imgs/**` and matches each local file to a resource by a "normalized stem" (lowercased, extension and trailing `_xxxxxx` hash removed,
   non-alphanumerics collapsed to `-`).
4. Writes `src/lib/generated/cloudinary-manifest.json` and prints a summary
   (counts of local assets, cloud assets, mapped, and missed).

Uploads

- **Server (signed):** use `src/lib/server/cloudinary.ts`.
  It configures the SDK only when full server config is present, and exposes `cloudinary`, `assertCloudinaryServerConfig()`, and `signCloudinaryParams()`.
- **Browser (unsigned):** `getUnsignedCloudinaryUploadUrl()` and `getUnsignedCloudinaryUploadFields()` from `cloudinary.ts` build a direct upload
  endpoint + form fields using the public API key and upload preset.

Local image optimization (separate from Cloudinary)

`npm run optimize-images` runs `scripts/optimize-images.js`,
which converts non-WebP images under `static/imgs` to WebP (quality 80) with `sharp`,
skipping files that are already WebP or up to date. This is independent of Cloudinary
and is about shrinking the locally-committed assets.

components

Components

Reusable components live in `src/lib/components` and are
imported with the `$lib` alias, e.g.:

```
import Navbar from '$lib/components/Navbar.svelte';
```

All components use **Svelte 5 runes** (`$props`, `$state`, `$derived`, `$bindable`).
Several use a `cn()` helper (`twMerge(clsx(...))`) to compose Tailwind classes.

Layout & navigation

`Navbar.svelte`

Top navigation with a mega-menu, language switcher (driven by `locale` / `locales`
from i18n, with `langLabels` for display names), and a "book" entry point that opens
the `BookingDrawer`. Uses the `clickOutside` action to close menus.

`Footer.svelte`

Site footer. Pure presentational; pulls all copy from the i18n store
(`$t('footer.*')`).

`CookieBanner.svelte`

Renders only while consent is unset (`$cookieConsent === null`). Calls `setCookieConsent('accepted' | 'rejected')` from `src/lib/consent.ts` and links to `/cookie-policy`. See Analytics & consent.

`CloudinaryRuntime.svelte`

Invisible runtime hook mounted in the root layout. When Cloudinary delivery is enabled, it rewrites `img[src]`, `source[src]`, and `poster` attributes to Cloudinary
URLs (and observes DOM mutations to catch dynamically added nodes). See Cloudinary.

`SecurityGuard.svelte`

Mounts global listeners that disable right-click, copy/cut, drag, and common DevTools/print keyboard shortcuts — a light content-protection measure for the hotel's imagery. (Note: this is a deterrent only, not real security.)

Booking

`BookingBar.svelte`

Horizontal booking widget (used on the home page). Local `$state` for `arrival`, `departure`, `adults`, `children`; opens the `Calendar` and a guests dropdown; builds the booking URL with `getBookingEngineUrl('booking_bar')`.

`BookingDrawer.svelte`

Slide-in booking panel (opened from the navbar). Prop: `isOpen` (`$bindable`, default `false`). Shows date/guest selectors plus direct contact actions (phone/WhatsApp/email). Builds the booking URL via `getBookingEngineUrl`.

`RoomBookingWidget.svelte`

Compact booking widget used on room pages. Same pattern as `BookingBar`, with
`getBookingEngineUrl('room_booking_widget')`.

`Calendar.svelte`

Reusable date-range picker. Props: `arrival` (`$bindable`), `departure` (`$bindable`), `onSelect` callback. Localizes month/day names from the active `locale` via a `localeMap` (e.g. `it` → `it-IT`). Disables past dates.

Carousels & galleries

`RoomCarousel.svelte`

Cycles through the rooms defined in `src/lib/rooms.ts`,
showing image, name, price, and details with prev/next controls.

`OffersCarousel.svelte`

Carousel of offer cards (title, tag, description, image, `href` to the offer page).
The offer list is defined inline in the component.

`PromoCarousel.svelte`

Marquee-style promo strip rendered globally in the layout. Each promo links to a booking-engine promotion via the helpers in
`src/lib/config/booking.ts` (RESTART, Torgnon Hiking,
Forte di Bard, Aosta Romana).

`ImageCarousel.svelte`

Generic image carousel. Props: `images: { src, alt }[]`, `aspectRatio` (default `aspect-[4/5]`), `autoPlay` (default `false`), `autoPlayInterval` (default `5000` ms). Uses Svelte `fade` transitions and arrow controls.

`HorizontalScrollJack.svelte`

Scroll-jacking horizontal gallery with lerp-smoothed scrolling and parallax layers.
Prop: `images: { src, alt }[]`.

`HorizontalScrollJackGSAP.svelte`

A simpler responsive grid variant of the gallery (despite the name, the committed version renders a CSS grid of lazy-loaded images). Prop: `images: { src, alt } []`.

Weather

`HomeWeatherWidget.svelte`

Compact current-conditions widget for the home page. Fetches `https://wttr.in/Torgnon?format=j1` on mount and renders temperature/wind/snow with Lucide icons.

`WeatherWidget.svelte`

Richer weather widget (temp, feels-like, wind, visibility, precipitation, snow) with an `iconFromCode` mapping. See Weather.

Forms

`OfferLeadForm.svelte`

Lead-capture block shown on offer pages. Props: `title`, `subtitle`, and optional `notesPlaceholder`, `submitLabel`, `successTitle`, `successText`. Surfaces direct booking contacts (phone `+39 379 335 7713`, email `booking@hotel-du-soleil.it`).

Component → feature map

Component	Primary doc
`BookingBar`, `BookingDrawer`, `RoomBookingWidget`, `Calendar`, `PromoCarousel`	Booking
`CloudinaryRuntime`	Cloudinary
`CookieBanner`	Analytics & consent
`HomeWeatherWidget`, `WeatherWidget`	Weather

Component	Primary doc
`Navbar`	i18n (language switcher)

configuration

Configuration reference

Every configuration file in the repo and what it controls.

`package.json`

Defines the project name (`hotel-du-soleil`), `"type": "module"` (ESM), the npm
scripts (see Development), and dependencies. Notable runtime deps:
`cloudinary`, `gsap`, `swiper`, `three` / `@types/three`, `lucide-svelte`,
`svelte-motion`, `clsx`, `tailwind-merge`.

`svelte.config.js`

```
import adapter from '@sveltejs/adapter-auto';
const config = {
	kit: { adapter: adapter() },
	vitePlugin: {
		dynamicCompileOptions: ({ filename }) =>
			filename.includes('node_modules') ? undefined
: { runes: true }
	}
};
```

- `adapter-auto` detects the deploy target at build time (Netlify in production). If you settle on a single host, you can swap in a specific adapter.
- **Runes mode** is forced on for project code (not `node_modules`), which is why components use `$state` / `$props` /etc.

`vite.config.ts`

```
export default defineConfig({ plugins: [tailwindcss(), sveltekit()] });
```

Tailwind CSS 4 is wired in via the `@tailwindcss/vite` plugin (no separate `tailwind.config.js`/PostCSS file — Tailwind 4 is configured in CSS).

`tsconfig.json`

Extends the generated `./.svelte-kit/tsconfig.json` and enables: `strict`, `allowJs` + `checkJs`, `esModuleInterop`, `forceConsistentCasingInFileNames`, `resolveJsonModule`, `skipLibCheck`, `sourceMap`, `moduleResolution: bundler`, and `rewriteRelativeImportExtensions`. Path aliases (other than `$lib`) are managed by SvelteKit.

`eslint.config.js`

Flat config composing: `includeIgnoreFile('.gitignore')`, `js.configs.recommended`, `typescript-eslint` recommended, `eslint-plugin-svelte` recommended, and `eslint-config-prettier` (+ the Svelte Prettier compat). Custom rules:

- `no-undef: 'off'` (recommended for TS projects).
- `svelte/no-navigation-without-resolve: 'off'`.

The Svelte block sets up the TS parser with `projectService` and the project's `svelteConfig` for `.svelte` / `.svelte.ts` / `.svelte.js` files.

`.prettierrc` / `.prettierignore`

Prettier options: **tabs**, single quotes, no trailing commas, 100-char width, plugins `prettier-plugin-svelte` + `prettier-plugin-tailwindcss`, with `tailwindStylesheet: ./src/routes/layout.css` so class sorting knows the

theme.
`.prettierignore` excludes lockfiles and `static/`.

`.npmrc`

```
engine-strict=true
```

Enforces the declared Node/npm engine constraints during install.

Tailwind theme — `src/routes/layout.css`

Tailwind 4 is configured **in CSS** here (imported by the root layout). This is where the alpine color palette (`alpine-text`, `alpine-gold`, `alpine-bg`, `alpine-border`,
`alpine-muted`, etc.), fonts, and global styles/animations are defined. The custom `is-visible` / `fade-up-element` classes used for scroll-reveal animations live here.

`netlify.toml`

```toml
[build]
  command = "npm run build"
  publish = "build"
[[redirects]]   # apex → www
[[redirects]]   # netlify.app → www
```

See Deployment.

Environment variables (`.env`)

Cloudinary-only; templated in `.env.example`. The app runs without them (falls back
to local assets). Full reference in Cloudinary. `.env` / `.env.*`
are git-ignored except `.env.example` and `.env.test`.

`app.html` (not a config file, but global setup)

Sets the document `lang` / `dir`, async-loads Google Fonts (Cormorant Garamond +
Inter), and bootstraps Google Analytics with Consent Mode (defaults denied). See Analytics & consent.

content

Content management

Most page copy is translatable text in the i18n JSON files (see Internationalization). This page covers the **structured content** that lives in code and the tooling used to author certain pages.

Room catalog — `src/lib/rooms.ts`

`src/lib/rooms.ts` exports the room data consumed by the rooms pages (`/camere`, `/camere/[slug]`) and `RoomCarousel`.

- `commonAmenities: string[]` — amenities shared by all rooms (TV, Wi-Fi, ski box,
  parking, half-board, etc.), spread into each room's `amenities`.
- `rooms: Record<string, Room>` — keyed by slug. Current keys: `matrimoniale`, `tripla`, `quadrupla_standard`, `familiare`.

Each room has:

```
{
  name: string;          // display name
  slug: string;          // matches the [slug] route param
  description: string;   // long description (may contain \n
paragraphs)
  price: string;         // e.g. 'da €120 / notte'
  size: string;          // e.g. '18–20 mq'
  occupancy: string;     // e.g. '2 ospiti'
  bedType: string;
  highlight: string;
```

```
  image: string;          // hero/card image path under /imgs/...
  amenities: string[];
  gallery: string[];      // image paths
}
```

> The `rooms` value is typed `Record<string, any>` — when adding fields, prefer
> tightening this to a proper `Room` interface.

Adding or editing a room

1. Add/edit an entry in `rooms` (the key is the URL slug).
2. Point `image` / `gallery` at files in `static/imgs/...` (and re-run `npm run optimize-images` / `npm run cloudinary:sync` if needed).
3. The `/camere/[slug]` page resolves content by slug automatically.

Offers / promotions

Offer **pages** live under `src/routes/offerte/<slug>/+page.svelte` and their booking deep-links/IDs live in `src/lib/config/booking.ts`.
The featured lists shown in `OffersCarousel` and `PromoCarousel` are defined inline
in those components. See Booking for the full add-an-offer checklist.

Backlog — `to-do.md`

`to-do.md` is a free-form content/feature backlog (e.g. translations still needed, images/content to add for various sections, legal pages). It is not wired into any tooling — it's a human checklist.

Content-generation scripts (`scripts/*.py`)

Several Python scripts were used to generate or patch specific experience/wellness
pages programmatically:

Script	Target
`fix_ciaspolate.py`	`/esperienze/ciaspolate`
`fix_yoga.py`, `fix_yoga2.py`, `write_yoga.py`	`/esperienze/yoga`
`write_guide.py`	`/esperienze/guide`
`write_massaggi.py`	`/wellness/massaggi`

These are **one-off authoring helpers**, run manually with Python (not part of the npm build). They write Svelte page content. Treat them as historical/utility scripts; day-to-day edits are made directly in the `.svelte` files and the i18n JSON.

Asset conventions

- Public images/videos go in `static/imgs/...` and are referenced as `/imgs/...`.
- Prefer **WebP** (run `npm run optimize-images` to convert).
- When using Cloudinary delivery, keep the manifest in sync (`npm run cloudinary:sync`). See Cloudinary.

deployment

Deployment

The site is built with SvelteKit's **`adapter-auto`** and deployed to **Netlify**.

Build

```
npm run build      # vite build → produces the deployable output
npm run preview    # serve the build locally to sanity-check
```

`adapter-auto` (configured in `svelte.config.js`) detects the host at build time and selects the appropriate adapter — Netlify in production. If you

ever pin to a single host, replace it with the host-specific adapter (e.g. `@sveltejs/adapter-netlify`).

Netlify configuration

`netlify.toml`:

```
[build]
  command = "npm run build"
  publish = "build"

[[redirects]]
  from = "https://hotel-du-soleil.it/*"
  to = "https://www.hotel-du-soleil.it/:splat"
  status = 301
  force = true

[[redirects]]
  from = "https://hoteldusoleil.netlify.app/*"
  to = "https://www.hotel-du-soleil.it/:splat"
  status = 301
  force = true
```

- **Build command:** `npm run build`; **publish dir:** `build`.
- **Canonical host redirects:** the apex domain (`hotel-du-soleil.it`) and the `*.netlify.app` URL are 301-redirected to `https://www.hotel-du-soleil.it`. The `www` host is the canonical one (also used in the sitemap and host redirects).

Environment variables in production

If Cloudinary delivery is used in production, set the `PUBLIC_CLOUDINARY_*` and `CLOUDINARY_API_SECRET` variables in the Netlify site settings (Build & deploy → Environment). Without them the site serves local `/imgs/...` assets. See Cloudinary and Development.

SEO / canonical URLs

- The canonical origin is `https://www.hotel-du-soleil.it`.
- `/sitemap.xml` is generated dynamically from the route list in `src/routes/sitemap.xml/+server.ts`.
- Keep the sitemap, the host redirects, and the hidden-section logic in `src/hooks.server.ts` consistent when changing which pages are public.

Pre-deploy checklist

```
npm run format    # apply Prettier
npm run lint      # prettier --check + eslint
npm run check     # svelte-check type-check
npm run build     # ensure a clean production build
```

development

Development guide

Prerequisites

- **Node.js** (project developed with Node 22; `package.json` uses ESM/ `"type": "module"`).
  `.npmrc` sets `engine-strict=true`, so an incompatible Node version will fail install.
- **npm** (a `package-lock.json` is committed; prefer `npm`).

Install

```
npm install
```

This also runs the `prepare` script (`svelte-kit sync`) to generate `.svelte-kit` types. If you ever see missing `$types` / `$env` types, re-run:

```
npm run prepare
```

Environment variables

Copy the example file and fill in the values you need:

```
cp .env.example .env
```

`.env.example` contains the Cloudinary configuration:

```
PUBLIC_CLOUDINARY_CLOUD_NAME=
PUBLIC_CLOUDINARY_API_KEY=
PUBLIC_CLOUDINARY_UPLOAD_PRESET=
PUBLIC_CLOUDINARY_ENABLE_DELIVERY=false
PUBLIC_CLOUDINARY_BASE_FOLDER=hotel-du-soleil
CLOUDINARY_API_SECRET=
```

- The site runs **fine without any of these** — when Cloudinary is not configured the app simply serves local `/imgs/...` files.
- `PUBLIC_*` vars are exposed to the browser; `CLOUDINARY_API_SECRET` is server-only.
- See Cloudinary media pipeline for the full meaning of each var.

`.env` and `.env.*` are git-ignored (except `.env.example` / `.env.test`).

Run the dev server

```
npm run dev
# or open a browser automatically:
npm run dev -- --open
```

Vite serves the app (default `http://localhost:5173`).

npm scripts

Script	Command	What it does
`dev`	`vite dev`	Start the dev server with HMR.
`build`	`vite build`	Production build.
`preview`	`vite preview`	Serve the production build locally.
`prepare`	`svelte-kit sync`	Generate SvelteKit types (runs on install).
`check`	`svelte-kit sync && svelte-check`	Type-check `.svelte` / `.ts` against `tsconfig.json`.
`check:watch`	same, `--watch`	Continuous type-checking.
`lint`	`prettier --check . && eslint .`	Verify formatting + lint.
`format`	`prettier --write .`	Auto-format the codebase.
`cloudinary:sync`	`node scripts/sync-cloudinary-manifest.mjs`	Rebuild the Cloudinary manifest (needs Cloudinary creds in `.env`).
`optimize-images`	`node scripts/optimize-images.js`	Convert `static/imgs` files to WebP.

Lint & format

The repo uses **Prettier** (formatting) and **ESLint flat config** (linting), and the `lint` script runs both. Before committing:

```
npm run format   # apply formatting
npm run lint     # verify formatting + lint rules
npm run check    # type-check
```

Formatting conventions (from `.prettierrc`): **tabs**, single quotes, no trailing commas, 100-char print width, with the Svelte and Tailwind Prettier plugins.

> There are **no pre-commit hooks** configured in this repo (no Husky / no `.pre-commit-config.yaml`), so run the checks above manually.

TypeScript

`tsconfig.json` extends the generated `.svelte-kit/tsconfig.json` and enables
`strict`, `checkJs`/`allowJs`, and `resolveJsonModule`. Use the `$lib` alias for library imports (e.g. `import { t } from '$lib/i18n'`).

Building & previewing

```
npm run build
npm run preview
```

Production deployment is handled by Netlify — see Deployment.

internationalization

Internationalization (i18n)

The site uses a small **custom translation system** built on Svelte stores — there is no external i18n library. It lives in `src/lib/i18n`.

How it works

`src/lib/i18n/index.ts` imports one JSON file per language and exposes:

Export	Type	Description
`locale`	`writable<string>`	The active language code. Defaults to `'it'`.
`t`	`derived` store	A translator: `$t('section.key')` returns the string for the current locale.
`dir`	`derived` store	Text direction — `'rtl'` for Arabic, otherwise `'ltr'`.
`locales`	`string[]`	The list of locales offered in the UI.

Usage in a component:

```
<script lang="ts">
	import { t, locale, locales } from '$lib/i18n';
</script>

<h1>{$t('home.hero.title')}</h1>
<button onclick={() => locale.set('en')}>EN</button>
```

Lookup & fallback

`t` splits the key on `.` and walks the nested JSON object. If a key is missing in the active locale (and the active locale isn't Italian), it **falls back to Italian (`it`)**. If still missing, it returns `undefined`.

```
export const t = derived(locale, ($locale) => (key: string) => {
	const keys = key.split('.');
	let value = translations[$locale];
	for (const k of keys) value = value?.[k];
	if (value === undefined && $locale !== 'it') {
		value = translations['it'];
		for (const k of keys) value = value?.[k];
	}
	return value;
});
```

Because values may be `undefined`, some pages render translated HTML with `{@html $t('...')}` (e.g. the hero title) — keep translation values trusted/static.

Direction (RTL)

The root layout reacts to `dir` and sets `document.documentElement.dir`:

```
// +layout.svelte
$effect(() => {
	document.documentElement.lang = $locale;
```

```
        document.documentElement.dir = $dir; // 'rtl' for ar
});
```

Locales

Translation files live in `src/lib/i18n/locales`:

Code	File	Notes
`it`	`it.json`	**Default & fallback** language (Italian).
`en`	`en.json`	English.
`ru`	`ru.json`	Russian.
`fr`	`fr.json`	French.
`de`	`de.json`	German.
`es`	`es.json`	Spanish.
`ar`	`ar.json`	Arabic (renders RTL).
`zh`	`zh.json`	Chinese.

> **Important:** all eight files are imported and available to `t`, but the UI's active language list is currently limited by `export const locales = ['it', 'en', 'ru'];` in `index.ts`. The `Navbar` builds its switcher from `locales`, so only those three are user-selectable by default even though more translation files exist. To expose another language, add its code to the `locales` array.

Key namespaces

Each locale JSON is organized into top-level sections, including:

```
common, nav, megamenu, hero, booking, home, footer,
rooms, rooms_data, rooms_amenities, rooms_specs,
ristorante, wellness, sport, experiences, struttura, posizione,
sostenibilita, meteo, eventi, kids_club,
```

```
offers_page, promo, restart_page, torgnon_offer_page,
aosta_romana_offer_page, forte_bard_offer_page, offer_lead_form,
degustazione_page, ciaspolate_page, yoga_page, guide_page,
massaggi_page,
policy, cookie_policy, cookie_banner, termini
```

Editing / adding translations

- **Edit copy:** find the key under the relevant section in `it.json` (and the other locale files) and update the value.
- **Add a new key:** add it to **every** locale file you support — at minimum `it.json`
  (the fallback). Missing keys silently fall back to Italian, then to `undefined`.
- **Add a new language:**
  1. Create `src/lib/i18n/locales/<code>.json`.
  2. Import it in `index.ts` and add it to the `translations` map.
  3. Add `<code>` to the `locales` array to show it in the switcher.
  4. If it's an RTL language, extend the `dir` logic accordingly.
  5. Add a display label in `Navbar.svelte`'s `langLabels`.

project-structure

Project structure

Annotated map of the repository. Generated/ignored folders (`node_modules`, `.svelte-kit`, `build`) are omitted.

```
Hotel-Du-Soleil/
├── README.md                  # Project overview + quick start
(links to docs/)
├── docs/                      # ← this documentation
├── to-do.md                   # Content/feature backlog (see
docs/content.md)
├── package.json               # Scripts and dependencies
├── package-lock.json
```

```
├── svelte.config.js          # SvelteKit + adapter-auto config
(runes enabled)
├── vite.config.ts            # Vite plugins (Tailwind +
SvelteKit)
├── tsconfig.json             # TypeScript (strict, extends
generated config)
├── eslint.config.js          # Flat ESLint config (JS/TS/Svelte
+ Prettier)
├── .prettierrc               # Prettier (tabs, single quotes,
Svelte+Tailwind plugins)
├── .prettierignore
├── .npmrc                    # engine-strict=true
├── .env.example              # Template for Cloudinary env vars
├── netlify.toml              # Build command, publish dir, host
redirects
├── index.html.bak            # Legacy static HTML (pre-
SvelteKit) — not used by the app
├── .vscode/                  # Editor settings
├── scripts/                  # Build/content tooling (see below)
├── static/                   # Static assets served at site root
(images, robots, etc.)
└── src/
    ├── app.html              # HTML document shell (fonts, GA
bootstrap)
    ├── app.d.ts              # Ambient TS types
(Window.gtag/dataLayer, App namespace)
    ├── hooks.server.ts       # Server hook: redirects hidden
sections to /
    ├── routes/               # File-based routes (pages,
layouts, endpoints)
    └── lib/                  # Reusable code, importable via the
`$lib` alias
```

src/lib

```
src/lib/
├── index.ts                  # Barrel: re-exports ./cloudinary
```

```
├── rooms.ts                    # Room catalog data (names, prices,
galleries, amenities)
├── analytics.ts                # GA4 event helpers + URL
classification
├── consent.ts                  # Cookie-consent store + Google
Consent Mode updates
├── cloudinary.ts               # Browser-safe Cloudinary URL
building + manifest lookup
├── assets/                     # In-bundle assets (favicon)
├── config/
│   └── booking.ts              # Booking engine URL + promotion
deep-link helpers
├── generated/
│   └── cloudinary-manifest.json   # Local-path → Cloudinary
public-id map (generated)
├── i18n/
│   ├── index.ts                # Locale store, `t` translator,
`dir`, locales list
│   └── locales/                # it, en, ru, fr, de, es, ar, zh
JSON files
├── server/
│   └── cloudinary.ts           # Server-only signed Cloudinary
operations (uses secret)
├── utils/
│   └── clickOutside.ts         # Svelte action for outside-click
detection
└── components/                 # Reusable UI components (see
docs/components.md)
```

`src/routes`

See Routes & pages for the full URL map. Top level:

```
src/routes/
├── +layout.svelte              # Global shell (navbar, footer,
analytics, etc.)
├── +page.svelte                # Home page
```

```
├── layout.css                # Tailwind theme + global styles
├── camere/                   # Rooms (list + [slug] detail)
├── ristorante/               # Restaurant (+ cantina, lounge-
bar, menu)
├── wellness/                 # Wellness (+ spa, private-spa,
massaggi, trattamenti)
├── sport/                    # Sport (+ bike, sci, trekking,
noleggio) — HIDDEN via hook
├── esperienze/               # Experiences (+ ciaspolate,
degustazione, guide, yoga)
├── offerte/                  # Offers/promotions (one folder per
package)
├── eventi/ , kids-club/ , storia/ , struttura/ , sostenibilita/
├── posizione/                # Location — HIDDEN via hook
├── meteo/                    # Weather page (+page.server.ts
loads Open-Meteo)
├── policy/ , cookie-policy/ , termini/   # Legal pages
└── sitemap.xml/+server.ts    # Dynamic sitemap endpoint
```

`scripts/`

Script	Run via	Purpose
`optimize-images.js`	`npm run optimize-images`	Convert images under `static/imgs` to WebP using `sharp`.
`sync-cloudinary-manifest.mjs`	`npm run cloudinary:sync`	Match local `static/imgs` files to Cloudinary resources and write `src/lib/generated/cloudinary-manifest.json`.
`fix_ciaspolate.py`, `fix_yoga.py`, `fix_yoga2.py`, `write_guide.py`, `write_massaggi.py`, `write_yoga.py`	run manually with Python	One-off content-authoring helpers used to generate/patch specific experience pages. See Content management.

Notable non-source files

- **`index.html.bak`** — a large legacy single-file HTML version of the site kept for reference. It is **not** part of the SvelteKit build.
- **`static/`** — anything here is served from the site root (e.g. `static/imgs/foo.webp` → `/imgs/foo.webp`). It is excluded from Prettier (see `.prettierignore`).

routes

Routes & pages

Routing is **file-based** under `src/routes`. Page URLs use Italian slugs (the site's primary language is Italian). This page lists every route, notes which ones are intentionally hidden, and documents the SEO endpoints.

Layouts & shell

- `+layout.svelte` — global wrapper applied to all pages (navbar, footer, cookie banner, promo carousel, analytics, chat widget, scroll-reveal observer). See Architecture.
- `layout.css` — Tailwind theme + global CSS, imported by the layout.

Page routes

URL	File	Notes
`/`	`+page.svelte`	Home: hero (ir optional Cloud video), bookin weather, room carousels.
`/camere`	`camere/+page.svelte`	Rooms overvie
`/camere/[slug]`	`camere/[slug]/+page.svelte`	Room detail; `s` matches a key `src/lib/room` (`matrimonial` `tripla` , `quadrupla_st` `familiare`).

URL	File	Notes
`/ristorante`	`ristorante/+page.svelte`	Restaurant lan
`/ristorante/menu`	`ristorante/menu/+page.svelte`	Menu.
`/ristorante/cantina`	`ristorante/cantina/+page.svelte`	Wine cellar / w
`/ristorante/lounge-bar`	`ristorante/lounge-bar/+page.svelte`	Lounge bar.
`/wellness`	`wellness/+page.svelte`	Wellness landi
`/wellness/spa`	`wellness/spa/+page.svelte`	Spa.
`/wellness/private-spa`	`wellness/private-spa/+page.svelte`	Private spa.
`/wellness/massaggi`	`wellness/massaggi/+page.svelte`	Massages.
`/wellness/trattamenti`	`wellness/trattamenti/+page.svelte`	Treatments.
`/esperienze`	`esperienze/+page.svelte`	Experiences la
`/esperienze/ciaspolate`	`esperienze/ciaspolate/+page.svelte`	Snowshoeing.
`/esperienze/degustazione`	`esperienze/degustazione/+page.svelte`	Tasting.
`/esperienze/guide`	`esperienze/guide/+page.svelte`	Guided activiti
`/esperienze/yoga`	`esperienze/yoga/+page.svelte`	Yoga.
`/struttura`	`struttura/+page.svelte`	The property/fa
`/storia`	`storia/+page.svelte`	History.
`/sostenibilita`	`sostenibilita/+page.svelte`	Sustainability.
`/eventi`	`eventi/+page.svelte`	Events.
`/kids-club`	`kids-club/+page.svelte`	Kids club.
`/meteo`	`meteo/+page.svelte` + `meteo/+page.server.ts`	Weather — se loaded from O Meteo. See W
`/offerte`	`offerte/+page.svelte`	Offers index. S offer rows belc
`/policy`	`policy/+page.svelte`	Privacy/websit
`/cookie-policy`	`cookie-policy/+page.svelte`	Cookie policy.
`/termini`	`termini/+page.svelte`	Terms & condi

Offers (`/offerte/*`)

Each promotion is its own page. Several deep-link into the booking engine via the promotion-URL helpers in `src/lib/config/booking.ts` (see Booking).

URL	File
`/offerte/restart`	`offerte/restart/+page.svelte`
`/offerte/torgnon-hiking-adventure`	`offerte/torgnon-hiking-adventure/+page.svelte`
`/offerte/forte-di-bard-gourmet-escape`	`offerte/forte-di-bard-gourmet-escape/+page.svelte`
`/offerte/aosta-romana-castello-di-fenis`	`offerte/aosta-romana-castello-di-fenis/+page.svelte`
`/offerte/workation-alpino`	`offerte/workation-alpino/+page.svelte`
`/offerte/family-base-camp`	`offerte/family-base-camp/+page.svelte`
`/offerte/family-mountain-camp`	`offerte/family-mountain-camp/+page.svelte`
`/offerte/summit-taste`	`offerte/summit-taste/+page.svelte`
`/offerte/wheels-relax`	`offerte/wheels-relax/+page.svelte`
`/offerte/orizzonti-silenzio`	`offerte/orizzonti-silenzio/+page.svelte`

Hidden routes (redirected)

`src/hooks.server.ts` intercepts requests and issues a `307` redirect to `/` for these paths, so the pages exist in the repo but are not publicly reachable:

- `/posizione`
- `/sport` and any `/sport/*` (`/sport/bike`, `/sport/sci`, `/sport/trekking`, `/sport/noleggio`)

```
function isHiddenSectionPath(pathname: string): boolean {
	return pathname === '/posizione' || pathname === '/sport'
|| pathname.startsWith('/sport/');
}
```

To re-enable a section, remove its path from `isHiddenSectionPath` (and, for SEO,
add it back to the sitemap below).

SEO endpoints

- **`/sitemap.xml`** — `sitemap.xml/+server.ts`
  returns an XML sitemap built from a hard-coded `routes` array against `https://www.hotel-du-soleil.it`. It sets `lastmod` to today's date and uses
  `priority` 1.0 for `/`, 0.9 for `/offerte`, and 0.8 for the rest.

  > Note: the array currently includes `/posizione` and `/sport` even though those
  > are redirected by the server hook — keep them in sync when changing visibility.

- **Per-page metadata** — pages set `<title>`, `<meta name="description">`, and
  keywords inside `<svelte:head>` (see the home page for an example).

Adding a page

1. Create `src/routes/<slug>/+page.svelte`.
2. Pull copy from the i18n store (`$t('...')`) and add the keys to each locale file under `src/lib/i18n/locales` (see i18n).
3. Add the URL to the sitemap `routes` array.
4. If the page should be reachable, make sure it isn't matched by `isHiddenSectionPath`.

weather

Weather feature

The site shows live mountain weather for **Torgnon** (the hotel's location) in two places: small widgets on marketing pages, and a full `/meteo` forecast page.

Coordinates used: lat `45.844`, lon `7.575`, altitude `1489 m`.

`/meteo` page (server-rendered, Open-Meteo)

`src/routes/meteo/+page.server.ts` loads the forecast **on the server** and passes normalized data to `src/routes/meteo/+page.svelte`.

- **Source:** Open-Meteo `https://api.open-meteo.com/v1/forecast`.
- **Requested data:** a rich set of `current`, `hourly` (incl. 80 m / 180 m layers and freezing-level height), and `daily` fields; `forecast_days=7`, `timezone=Europe/Rome`.
- **Resilience:** the fetch is wrapped in an `AbortController` with a **4.5 s timeout**. On any error/timeout it returns a safe payload with `weather: null` and a status message rather than failing the page.
- **Caching:** sets `cache-control: public, max-age=600, s-maxage=600, stale-while-revalidate=1200` (10-minute cache).

Returned `data` shape

```
{
  resortName: 'Torgnon',
  lastUpdated: string | null,           // ISO timestamp
  weather: NormalizedWeather | null,    // current conditions +
base/mid/upper layers
  dailyForecast: NormalizedDailyForecast[],  // up to 7 days
  officialLinks: { title, label, url }[],    // Torgnon ski area
+ Panomax webcam
  status: { configured, liveForecast, liveOperations, message }
}
```

Normalization helpers (in `+page.server.ts`)

- `toNumber()` — safe numeric coercion (returns `null` for empty/non-finite).
- `windDirectionFromDegrees()` — degrees → `N/NE/E/.../NW`.

- `weatherDescription()` / `weatherIcon()` — map WMO `weather_code` to an Italian
  description and an icon key (`sun`, `partly-cloudy`, `fog`, `rain`, `snow`, `snow-showers`, `thunderstorm`, ...).
- `normalizeForecast()` — builds current conditions plus three altitude **layers** (`base` from surface, `mid` from 80 m hourly, `upper` from 180 m hourly).
- `normalizeDailyForecast()` — maps the daily arrays to a typed per-day list.

> Snowfall is reported by Open-Meteo in mm; the loader converts it to cm (`freshSnowCm = snowMm / 10`).

Home & section widgets (client-side)

These fetch directly from the browser on mount:

Component	Source	Notes
`HomeWeatherWidget.svelte`	`https://wttr.in/Torgnon?format=j1`	Compact current conditions on the home page.
`WeatherWidget.svelte`	(weather-code based)	Richer widget with temp, feels-like, wind, visibility, precipitation, snow; has an `iconFromCode` mapping.

> Note there are **two weather data sources** in the codebase: the `/meteo` page uses
> **Open-Meteo** (server-side, full forecast), while `HomeWeatherWidget` uses **wttr.in** (client-side, quick current conditions). Keep this in mind when changing providers.

External references shown on `/meteo`

- **Torgnon Ski Area** — piste/lift status: https://www.torgnon.net/inverno/
- **Panomax Torgnon** — live 360° webcam: https://torgnon-skiarea.panomax.com/

`liveOperations` is hard-coded `false` — lift/piste status is **not** scraped; the page links out to the official sources instead.